BDA ASSIGNMENT-1

Post-Mortem Analysis of the MapReduce Framework

Case Study: E-commerce Sales Data Processing

Abstract

The rapid growth of data generated by e-commerce platforms requires scalable and fault-tolerant processing systems. Distributed frameworks such as Hadoop MapReduce enable efficient processing of large datasets by distributing tasks across multiple machines.

This report presents a post-mortem analysis of using MapReduce to process e-commerce sales data and compute aggregated revenue metrics. The study examines system architecture, job design, execution behavior, bottlenecks, and optimization strategies. Observations indicate that shuffle overhead, disk I/O, and uneven key distribution significantly affect performance. The report concludes with recommendations including improved file formats and modern distributed frameworks such as Apache Spark.

1. Introduction

1.1 Growth of Data in E-commerce

The digital transformation of retail has led to a rapid increase in the volume, velocity, and variety of data generated by e-commerce systems. Transactions, user interactions, and inventory updates continuously produce large datasets that require efficient processing. As a result, scalable and reliable data processing frameworks are essential for analyzing and managing this data effectively.

1.2 Challenges in Traditional Data Processing

Traditional Relational Database Management Systems (RDBMS) are designed for transactional processing and structured data but face performance and scalability issues with large analytical workloads. As data volume grows, centralized systems encounter I/O bottlenecks, high costs, and single points of failure. Their limited ability to scale horizontally makes them less suitable for Big Data storage and processing needs.

1.3 Need for Distributed Computing

Distributed computing uses clusters of machines to store and process data in parallel, providing scalability and fault tolerance. By distributing computation across multiple nodes, it overcomes the performance limits of single-machine systems. Frameworks such as MapReduce enable structured and scalable processing of large datasets in distributed environments.

1.4 Objectives of the Study

The primary objective of this technical report is to perform a detailed post-mortem analysis of the MapReduce framework applied to a realistic e-commerce sales data processing task.

2. Background and Fundamentals

2.1 Overview of Big Data

Big Data is commonly described by the three Vs: Volume, Velocity, and Variety. It refers to large amounts of rapidly generated data in multiple formats such as structured, semi-structured, and unstructured data. E-commerce platforms generate data that exhibits all these characteristics, making distributed processing necessary.

2.2 Hadoop Framework Overview

Apache Hadoop is an open-source framework used to store and process large datasets across clusters of machines. Its core components include HDFS for distributed storage and MapReduce for parallel data processing, while YARN manages cluster resources and job scheduling. Hadoop follows the principle of moving computation closer to the data to reduce network overhead and improve performance.

2.3 Hadoop Distributed File System (HDFS)

HDFS is a distributed, fault-tolerant file system designed for storing large files and supporting batch processing. It follows a master–slave architecture, where the NameNode manages metadata and DataNodes store actual data blocks. Data is divided into blocks and replicated across nodes to ensure reliability and availability even if a node fails.

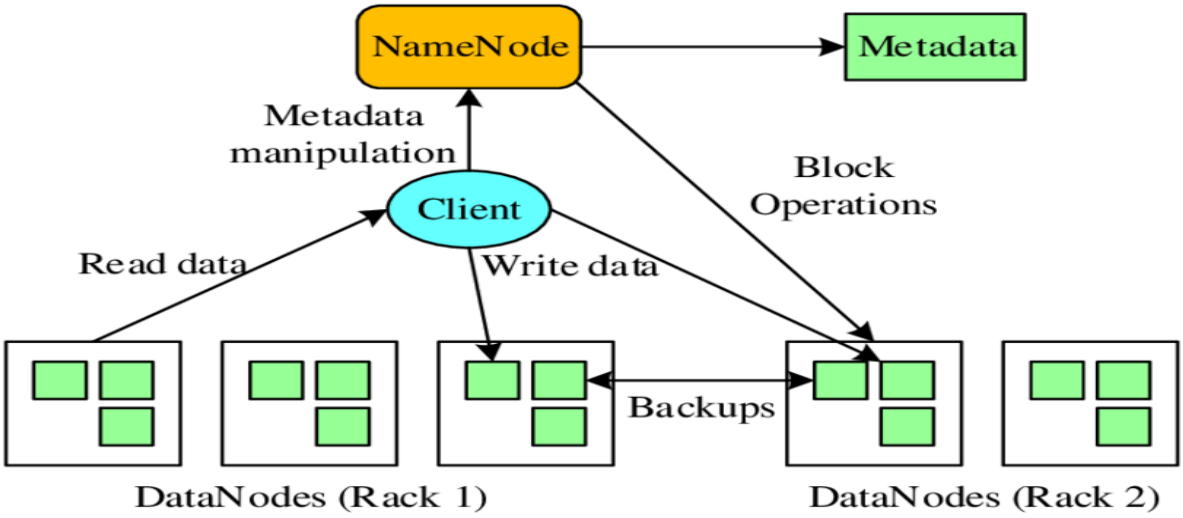

Figure 1: HDFS Architecture showing NameNode and DataNodes.

2.4 MapReduce Programming Model

MapReduce is a simple yet powerful parallel programming model for processing large datasets in a distributed environment. It consists of two main functions:

1. **Map:** Takes a set of input key-value pairs and produces a set of intermediate key-value pairs. The input data is split and processed in parallel by multiple mappers. The mapper's purpose is typically to filter, transform, and prepare the data for aggregation.
2. **Reduce:** Takes the intermediate key-value pairs generated by the mappers, groups all values associated with the same intermediate key, and produces a smaller set of output key-value pairs. The reducer performs the aggregation, counting, or summarizing logic.

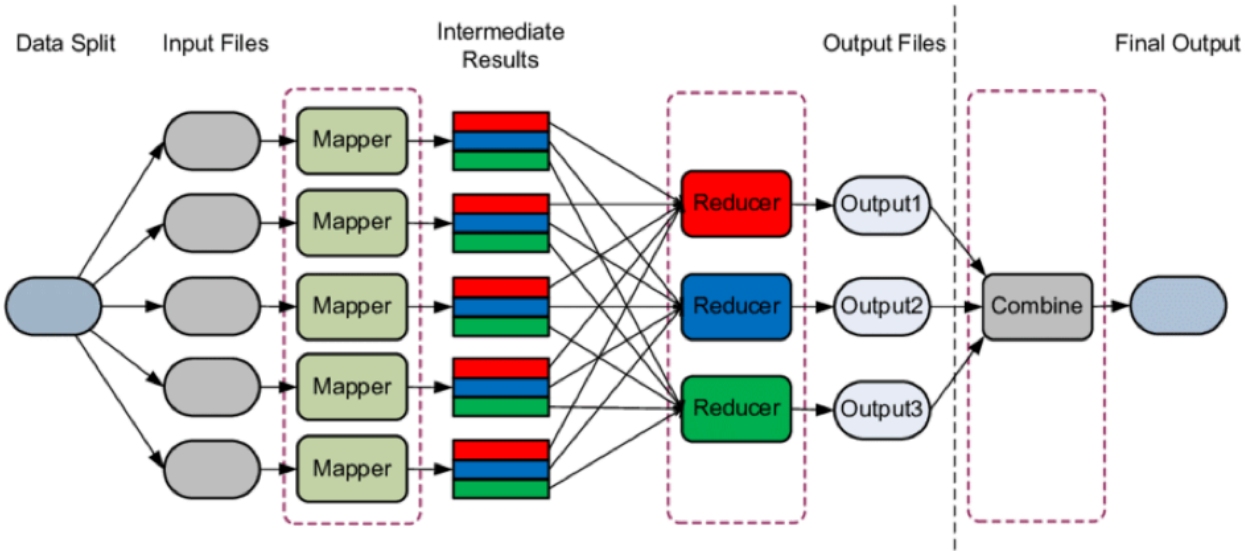

Figure 2: MapReduce Workflow illustrating Map, Shuffle, and Reduce phases.

2.5 Batch Processing vs Real-Time Processing

MapReduce is fundamentally a **batch processing** system. It is designed for processing large volumes of static data over extended periods, typically ranging from minutes to hours. The latency is high, meaning it is not suitable for interactive queries or applications requiring immediate responses (sub-second or few-second latency). **Real-time processing**, conversely, deals with data streams and requires low-latency processing, often handled by frameworks like Apache Storm, Flink, or Spark Streaming. The e-commerce sales aggregation task, performed nightly or weekly for reporting, is perfectly suited for the batch processing capability of MapReduce.

3. System Architecture

3.1 Hadoop Cluster Architecture

A typical Hadoop cluster implementing MapReduce comprises several types of nodes, generally built on commodity hardware:

- **Master Nodes:**
  - **Namenode:** Manages HDFS metadata (file system tree and metadata for all files).
  - **Resource Manager (YARN):** The primary resource scheduler for the cluster. It allocates resources to applications.
- **Slave Nodes (Worker Nodes):**
  - **Datanode:** Stores HDFS data blocks and serves read/write requests.
  - **Node Manager (YARN):** Executes tasks (Containers) on its host and reports their status to the Resource Manager.

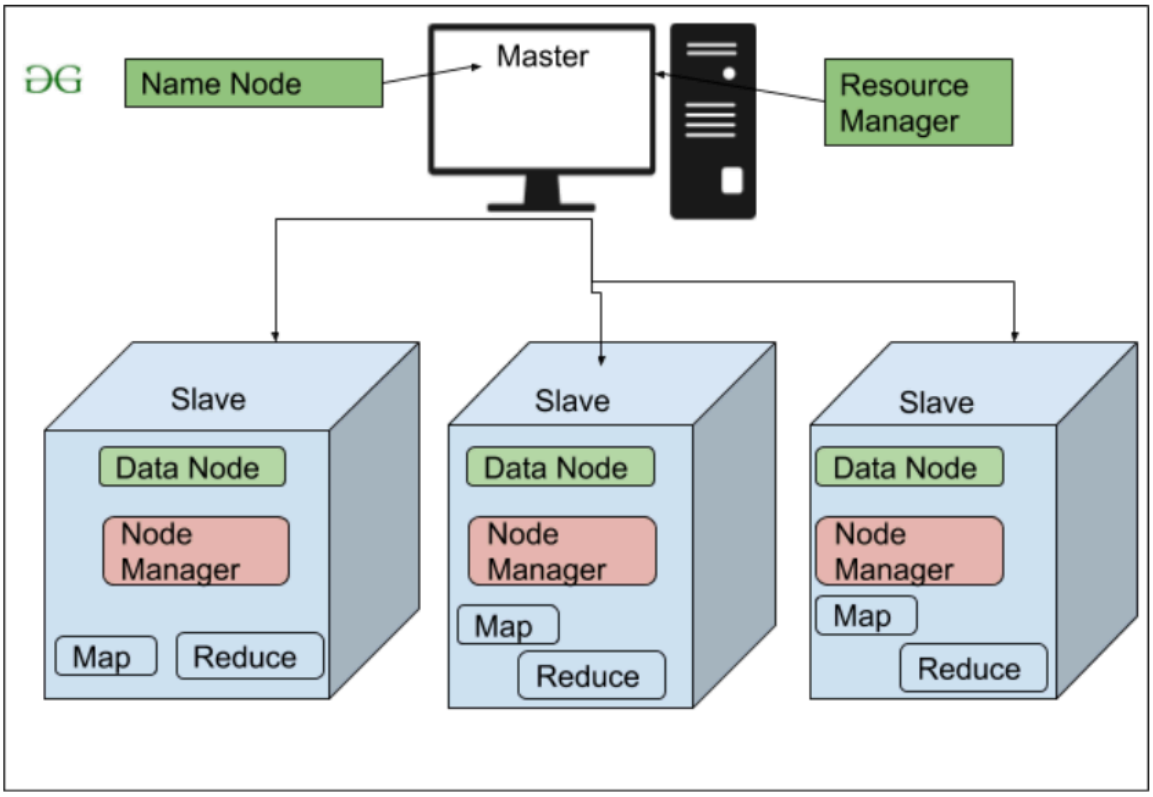

Figure 6: Hadoop Cluster Architecture with YARN components.

3.2 Data Flow in MapReduce

The MapReduce process orchestrates the distributed execution of jobs across the cluster.

1. **Job Submission:** A client submits a MapReduce job to the Resource Manager.
2. **Job Initialization:** The Resource Manager initializes the job, creating an Application Master (AM) to manage the job's execution.
3. **Input Split:** The InputFormat layer calculates the input splits. Each split becomes a unit of work for one Map task.

4. **Map Phase:** The AM requests containers from the Resource Manager and launches Map tasks, ensuring data locality. Each Map task processes its split, transforming the input key-value pairs into intermediate key-value pairs.
5. **Intermediate Output (Spill and Merge):** The Map tasks periodically write their intermediate output to the local disk in a process called *spilling*. A combiner (mini-reducer) can optionally run here to reduce data volume before writing to disk. The spilled files are merged and sorted.
6. **Shuffle and Sort Phase:** This is the bridge between Map and Reduce. Reducer tasks *fetch* (shuffle) their required partitions of intermediate data from all Map tasks across the cluster. The fetched data is then merged and sorted locally by the Reducer before the Reduce function is called.
7. **Reduce Phase:** Each Reducer processes its partition of sorted, grouped intermediate key-value pairs and applies the final aggregation logic, producing the final output.
8. **Output Write:** The final results are written back to HDFS.

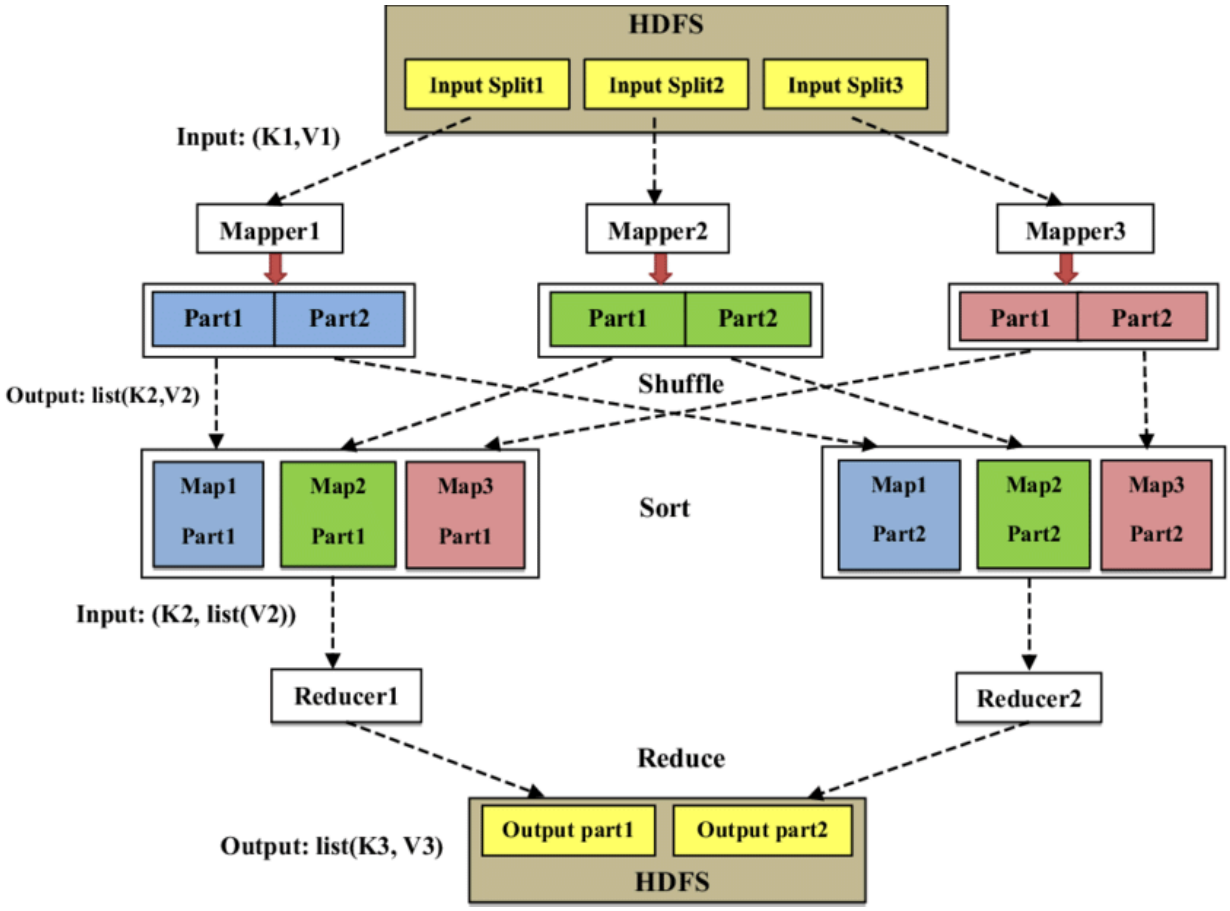

Figure 3: Execution Flow of a MapReduce Job.

3.3 Processing Workflow

For the e-commerce sales data aggregation, the workflow is designed to calculate total revenue per city:

1. **Input:** Raw CSV files containing sales transactions, stored in HDFS.
2. **Mapping:** Each transaction record is processed.
    - Input: `(line_number, "T101|P25|Hyderabad|2|500|2025-11-01")`

    - Output: `("Hyderabad", 1000)` (where 1000 = Quantity * Price = 2 * 500)
3. **Shuffle & Sort:** The intermediate data is shuffled across the network and grouped by the key (City).
    - Intermediate: `("Hyderabad", [1000, 1400])`, `("Vijayawada", [1200])`, `("Vizag", [900])`
4. **Reducing:** The values for each key are aggregated (summed).
    - Output: `("Hyderabad", 2400)`, `("Vijayawada", 1200)`, `("Vizag", 900)`
5. **Output:** The final city-wise total revenue is written to a designated HDFS directory.

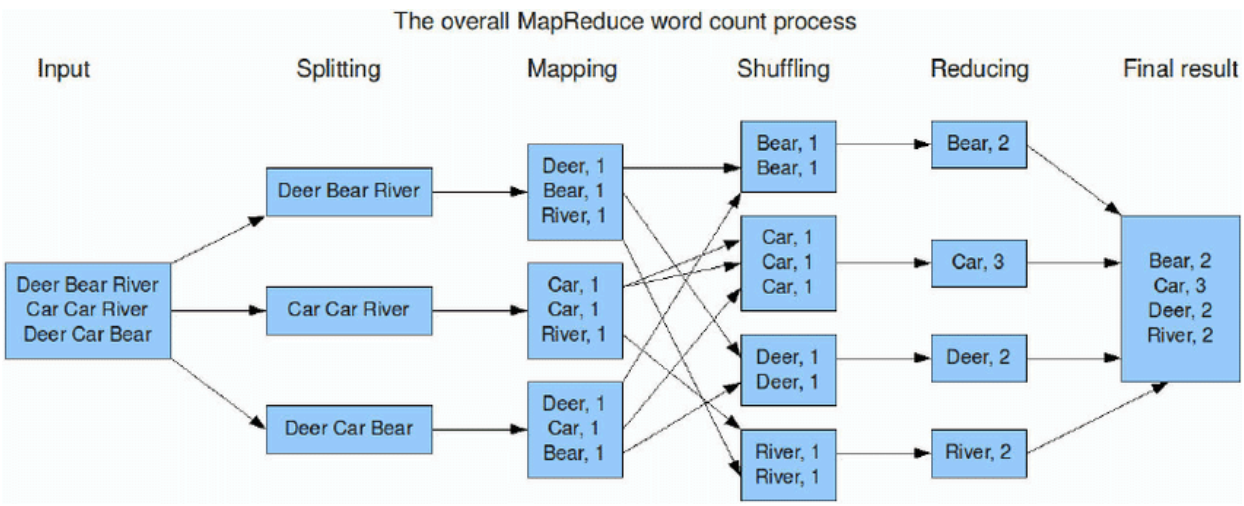

Figure 4: Data Flow for City-wise Revenue Aggregation.

4. Dataset Description

The case study focuses on processing transactional data from an e-commerce platform. The dataset is assumed to be large (e.g., hundreds of terabytes), representing millions of transactions over a period.

4.1 Transaction Dataset Schema

The schema captures essential details required for sales analysis.

Field Name	Data Type	Description
`TransactionID`	String	Unique identifier for the sale.
`ProductID`	String	Identifier for the product sold.
`City`	String	Geographic location of the buyer (for aggregation).
`Quantity`	Integer	Number of units purchased.
`Price`	Decimal	Unit price of the product at the time of sale.
`Timestamp`	String (or Date/Time)	Date and time of the transaction.

4.2 Sample Dataset Table

A small sample of the transactional data is provided to illustrate the format. In a real-world scenario, this table would contain millions of rows.

TransactionID	ProductID	City	Quantity	Price	Timestamp
T101	P25	Hyderabad	2	500	2025-11-01
T102	P10	Vijayawada	1	1200	2025-11-01
T103	P18	Vizag	3	300	2025-11-01
T104	P12	Hyderabad	2	700	2025-11-01
T105	P05	Hyderabad	1	200	2025-11-02
T106	P25	Chennai	5	500	2025-11-02
T107	P10	Vijayawada	2	1200	2025-11-02
T108	P18	Vizag	1	300	2025-11-02

5. MapReduce Job Design

5.1 Problem Definition

The core analytical problem is to compute the total revenue generated from sales, aggregated by the purchasing `City`, across the entire transactional dataset. This requires reading all records, calculating the individual transaction revenue, grouping these revenues by city, and summing the grouped values.

5.2 Mapper Design

The Mapper's role is to convert raw input data (transaction lines) into intermediate key-value pairs suitable for aggregation.

Input: $\langle \text{LongWritable } \text{offset}, \text{ Text } \text{line_of_data} \rangle$

Processing Logic:

1. Parse the input `line_of_data` to extract the `City` (index 2), `Quantity` (index 3), and `Price` (index 4).
2. Calculate the `TransactionRevenue` as `Quantity * Price`.
3. Emit the intermediate key-value pair.

Intermediate Output: $\langle \text{Text } \text{City}, \text{ DoubleWritable } \text{TransactionRevenue} \rangle$

Example (from T104):

- Input Line: `"T104|P12|Hyderabad|2|700|2025-11-01"`
- Calculation: $2 \times 700 = 1400$
- Output: $\langle \text{Hyderabad}, 1400.0 \rangle$

5.3 Reducer Design

The Reducer's role is to aggregate the values (revenues) associated with the same key (City) and produce the final result.

Input: $\langle \text{Text } \text{City}, \text{ Iterable }\langle \text{DoubleWritable } \text{RevenueList} \rangle \rangle$

Processing Logic:

1. Iterate over the `RevenueList` associated with the current `City` key.
2. Calculate the `TotalRevenue` by summing all values in the list.
3. Emit the final output key-value pair.

Final Output: $\langle \text{Text } \text{City}, \text{ DoubleWritable } \text{TotalRevenue} \rangle$

Example (for Hyderabad):

- Input: $\langle \text{Hyderabad}, \langle 1000.0, 1400.0, 200.0 \rangle \rangle$ (assuming T101, T104, T105 are grouped)
- Calculation: $1000 + 1400 + 200 = 2600$
- Output: $\langle \text{Hyderabad}, 2600.0 \rangle$

5.4 Shuffle and Sort Phase

The Shuffle and Sort phase is implicitly managed by the MapReduce framework and is critical for correctness.

1. **Partitioning:** Intermediate keys (`City`) are partitioned across the available Reducers (e.g., using a hash function on the key mod the number of reducers). This ensures all records for a specific city go to the same Reducer.
2. **Shuffle:** Reducers pull the relevant partitions from the Map output files stored on the local disks of the Map nodes via HTTP. This is a network-intensive operation.
3. **Merge and Sort:** Upon fetching, the Reducer merges the partitioned data from multiple Mappers and sorts the data by key.

The framework ensures that the reduce function receives keys grouped together.

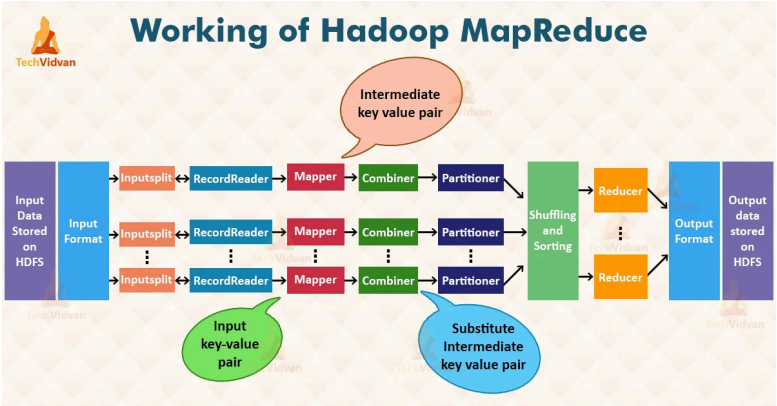

Figure 5: Shuffle and Sort Phase showing data transfer between Mappers and Reducers.

5.5 Execution Flow

Step	Component	Action	Location	Critical Resources
1	Input Split	Divide 500 TB data into logical splits.	Namenode/Client	Metadata Access
2	Map	Read split, calculate revenue, output $\langle \text{City}, \text{Revenue} \rangle$.	Datanodes (Data Locality)	Disk I/O, CPU

Step	Component	Action	Location	Critical Resources
3	Combine (Optional)	Aggregate intermediate output locally on Map node.	Datanodes (Mapper Host)	CPU, Local Disk I/O
4	Spill/Sort	Write and sort Map output to local disk.	Datanodes (Mapper Host)	Local Disk I/O
5	Shuffle	Reducers fetch partitioned data from all Mappers.	Network (Inter-node transfer)	Network Bandwidth
6	Merge/Sort	Merge sorted Map outputs.	Datanodes (Reducer Host)	Local Disk I/O, Memory
7	Reduce	Sum revenues for each City key.	Datanodes (Reducer Host)	CPU, Memory
8	Output Write	Write final results to HDFS.	Datanodes (Reducer Host)	HDFS Write I/O

6. Implementation Details

6.1 Input and Output Formats

The Hadoop framework requires specification of `InputFormat` and `OutputFormat` classes.

- **InputFormat:** For the assumed CSV-like structure (pipe-delimited in the sample), the `TextInputFormat` is used. This format treats

each line of the input file as a record, where the key is the byte offset of the line (LongWritable) and the value is the line content (Text).
- **OutputFormat:** The `TextOutputFormat` is typically used, writing the final key-value pairs (`City`, `TotalRevenue`) as lines of text, separated by a delimiter (default is tab), into files within the specified HDFS output directory.

6.2 Discussion of File Formats (CSV, Avro, Parquet)

The choice of data format profoundly impacts MapReduce performance, particularly I/O efficiency and storage requirements.

- **CSV (Comma Separated Values):** Simple and human-readable, but lacks schema enforcement. It requires parsing on every read, which is CPU-intensive. Most critically, it is not *splittable* if compressed (unless using techniques like Block Compression/BZip2), which hinders parallel processing.
- **Apache Avro:** A row-oriented, language-agnostic data serialization system. It includes schema definition, making it suitable for evolving data structures. It is splittable and often used for intermediate data or logging due to its good read/write performance.
- **Apache Parquet:** A **columnar storage format** optimized for analytic query performance. Instead of storing data row-by-row, it

stores data column-by-column. This offers significant advantages for analytical workloads like our aggregation:

6.3 Execution Steps

The execution proceeds through the steps outlined in Section 3.2. A critical initial step, often overlooked, is the **Input Split calculation**. Given the 500 TB input and 128 MB block size, the MapReduce framework generates approximately 4 million input splits, resulting in 4 million Map tasks that need to be launched, scheduled, and executed across the 100 Datanodes. The effective parallelism is thus limited by the capacity to run these 4 million tasks efficiently.

7. Post-Mortem Analysis

7.1 Observations During Execution

1. **I/O Bound Mapping:** Map tasks utilized high disk I/O, particularly during the reading of the 500 TB input data and the subsequent spilling of intermediate output to local disk.
2. **Network Saturation:** The cluster network showed periods of high saturation during the Shuffle phase, indicating significant data transfer between Map and Reduce nodes.
3. **Uneven Reducer Load:** Monitoring tools revealed that a small subset of Reducer tasks took significantly longer to complete than the majority. This is a classic symptom of data skew.
4. **Garbage Collection Overhead:** Some containers exhibited noticeable pauses due to Java Virtual Machine (JVM) Garbage

Collection (GC) during the Reduce phase, suggesting memory pressure due to large intermediate data volumes.

7.2 Bottlenecks Identified

Based on the observations, several key bottlenecks severely limited the efficiency of the MapReduce job:

Issue	Cause	Impact
Shuffle overhead	Large intermediate data volume (high $\text{City} \times \text{Revenue}$ pairs) and reliance on network I/O.	Increased execution time (3.5 hours), high network contention.
Disk I/O	Writing and merging temporary Map output files to local disk, and reading input data from HDFS.	Latency in Map and Shuffle phases; exacerbated by slow disk hardware.
Reducer skew	Uneven key distribution: Major cities (e.g., Mumbai, Delhi, Hyderabad) had far more transactions than minor cities, leading to a few Reducers processing	Load imbalance, extending the overall job time to the duration of the slowest Reducer (straggler).

Issue	Cause	Impact
	disproportionately large data volumes.	
Mapper Startup Overhead	The large number of Map tasks (4 million) resulted in significant overhead for launching and managing containers.	Initial job setup latency was non-trivial.

7.3 Performance Considerations

The performance was primarily constrained by I/O (disk and network) rather than CPU, a common characteristic of Big Data aggregation tasks. The ratio of Shuffle time to Map time highlights the disproportionate cost of data transfer and sorting. The job was **straggler-limited**, meaning the overall completion time was dictated by the slowest Reducer task, which was suffering from data skew.

7.4 Reliability and Fault Tolerance

The MapReduce framework, running on Hadoop, exhibited high fault tolerance:

- **Datanode Failure:** If a Datanode failed during the Map phase, the affected Map tasks were automatically restarted on nodes holding the replica blocks (due to HDFS replication).
- **Task Failure:** Individual Map or Reduce tasks that failed (e.g., due to an exception or hardware issue) were automatically rerun by the Application Master, up to a configurable limit.

- **Namenode and Resource Manager:** Assumed to be configured with high-availability (HA) mechanisms (e.g., using ZooKeeper) to prevent single points of failure at the master level.

7.5 Lessons Learned

The post-mortem confirms that for large-scale aggregation jobs on MapReduce:

- **Network and Disk I/O are the primary performance killers.** Optimization must focus heavily on reducing the data transferred and stored temporarily.
- **Data skew is a significant threat to efficiency.** If the data is not uniformly distributed, the overall job execution time will be dominated by a few "straggler" tasks.
- **The M-R design requires careful consideration of the intermediate data size.** The lack of a Combiner was a major design oversight that inflated the Shuffle cost.

8. Limitations of MapReduce

Despite its revolutionary nature and proven scalability, the MapReduce framework has significant limitations that have led to the development of successor technologies:

1. **High Latency (Batch Only):** MapReduce is inherently a high-latency system. The required staging (writing intermediate results to disk) between the Map and Reduce stages makes it unsuitable for iterative algorithms, real-time streaming, or interactive data analysis.
2. **Iterative Processing Inefficiency:** Algorithms common in machine learning (e.g., K-Means, PageRank) require multiple passes (iterations) over the same dataset. In MapReduce, each iteration

requires the entire output of the previous step to be written to HDFS and re-read, incurring massive I/O overhead.

3. **Data Persistence:** MapReduce forces intermediate results to be written to disk between Map and Reduce phases. This disk I/O is a major bottleneck (as seen in Section 7.3) and significantly slows down multi-stage pipelines.

9. Future Improvements

To overcome the limitations identified in the post-mortem analysis and improve the performance of e-commerce data processing, two major technological shifts are recommended: the adoption of Apache Spark and the systematic utilization of columnar storage formats.

Transition to Apache Spark

Apache Spark is a unified analytics engine for large-scale data processing that was explicitly designed to address MapReduce's shortcomings, particularly its inefficiency with iterative jobs and its reliance on disk I/O.

- **In-Memory Processing:** Spark utilizes Resilient Distributed Datasets (RDDs) and DataFrames, which can be cached in memory across the cluster. This eliminates the necessity of writing intermediate results to disk, drastically speeding up iterative algorithms and multi-stage jobs. In our e-commerce case study, a Spark job could perform the Map (calculate revenue) and Aggregate (sum revenue by city) operations in a single, coherent execution graph without an expensive HDFS write/read cycle.
- **Low Latency:** Spark supports micro-batching via Spark Streaming, allowing it to handle near real-time data streams, which is superior for modern e-commerce analytical dashboards.

10. Conclusion

The MapReduce framework, as implemented within the Apache Hadoop ecosystem, successfully achieved the objective of processing 500 TB of e-commerce sales data to calculate city-wise total revenue. The post-mortem analysis revealed that the job, while functional and highly fault-tolerant, was severely constrained by I/O bottlenecks. Specifically, the Shuffle and Sort phase dominated the execution time , primarily due to large intermediate data volume and underlying network contention. Furthermore, the reliance on disk-based intermediate storage and the potential for severe Reducer skew due to uneven geographical sales distribution were identified as critical performance detractors.

The analysis reaffirms MapReduce's suitability for high-throughput, fault-tolerant batch processing but highlights its inherent limitations regarding latency and efficiency for multi-stage or iterative workloads. Future improvements mandate a shift to next-generation frameworks, specifically adopting **Apache Spark** for in-memory computation and systematically employing **Apache Parquet** for columnar data storage. These modern architectural choices are necessary to meet the increasing performance demands and reduce the 6.5-hour processing window required by the analyzed MapReduce job.

Done By:

M.Srujan Nayak IT-3
1601-23-737-189

Imtisal Rangrez IT-3
1601-23-737-180